YONGE
JEWELRY

WEBSITE SETUP GUIDE

Putting the Yonge Jewelry website online

Step by step, with no coding: GitHub Pages shows the website, Square takes the payments, and a small free Cloudflare Worker connects the two.

What you will have at the end

- The new shop live at your web address
- Customers paying by card through Square, with the order in your Square Dashboard
- Pieces switching to Sold by themselves when online sales use up the stock
- "See it in store" and the text-to-book buttons working

What you need

- The file **yonge-jewelry-site.zip**
- The GitHub account that runs the site, and its repository
- GitHub Desktop (free) and a code editor such as VS Code (free)
- The shop's Square login
- An email address for a free Cloudflare account

The six parts

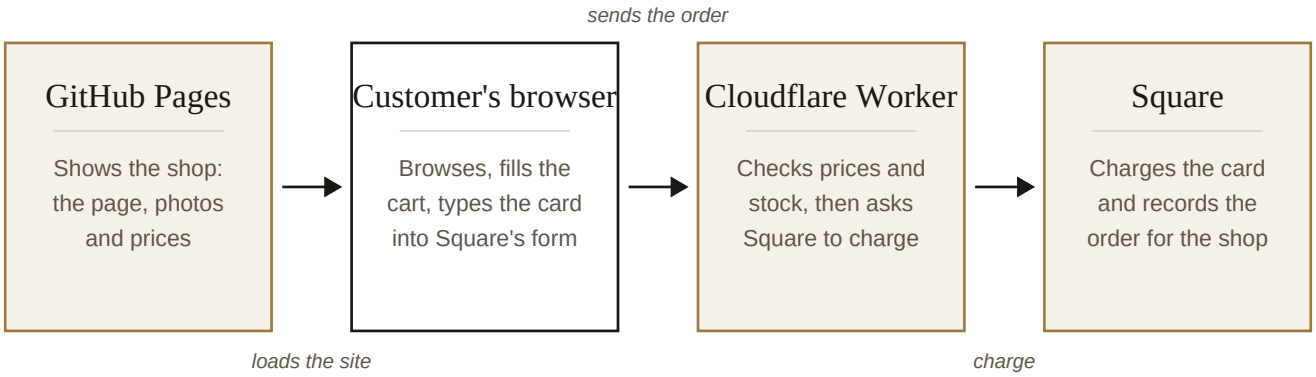
	Part	Time
1	Put the website on GitHub Pages	15 min
2	Create the Square app	10 min
3	Set up the payment helper on Cloudflare	15 min
4	Connect the website to Square	5 min
5	Test with a practice card	10 min
6	Go live with real payments	10 min

Do the parts in order; each one ends with a check so you know it worked. GitHub Pages, the Cloudflare Workers free plan and the Square developer app cost nothing. Square takes its usual card processing fee on each sale. Prepared October 13, 2026: screens on GitHub, Square and Cloudflare change from time to time, so if a button has a slightly different name, look for the closest match.

BEFORE YOU START

How the pieces fit together

Three services work together. **GitHub Pages** shows the website. **Square** takes the money. A small **Cloudflare Worker** sits between them: it double-checks prices and stock, then asks Square to charge the card. It is needed because GitHub Pages can only show files. It cannot run code or keep a password safe.



What is in the zip

File or folder	What it is	Where it goes
`website`	The shop: index.html, products.json (the catalogue), the photos and a hidden file called .nojekyll	Your GitHub repository (Part 1)
`payments-worker.js`	The payment helper	Pasted into Cloudflare (Part 3)
`Website-Setup-Guide.pdf`	This guide	Keep it handy

Important. Your Square **access token** works like a password to the shop's money. It goes in exactly one place: Cloudflare's settings, saved as a Secret (Part 3). Never paste it into index.html, GitHub, an email or a chat. If it ever leaks, replace it in the Square Developer Console.

Tip. Everything starts in Square's **sandbox**, a practice mode with test cards where no real money moves. You switch to real payments in Part 6, once everything works.

PART 1 OF 6 · ABOUT 15 MINUTES

Put the website on GitHub Pages

You will replace the old site files in your GitHub repository with the new ones, then turn on GitHub Pages. GitHub Desktop makes this easy. The site has about 240 files and the GitHub website only uploads 100 at a time.

1. **Unzip the file**

   Double-click **yonge-jewelry-site.zip**. A folder opens with **website**, **payments-worker.js** and this guide inside.

2. **Install GitHub Desktop and sign in with the site account**

   Download it from **desktop.github.com** and open it. If it is signed in to a different account, go to **GitHub Desktop › Settings › Accounts** (on Windows: **File › Options › Accounts**), click **Sign out**, then sign in with the account that owns the website repository.

3. **Download (clone) the website repository and open its folder**

   Click **File › Clone Repository…**, pick the site's repository from the list, choose where to save it, and click **Clone**. Then click **Repository › Show in Finder** (on Windows: **Show in Explorer**).

4. **Remove the old site files**

   Delete the old files and folders, but keep two things: the **CNAME** file if there is one (it holds your web address, such as yongejewelry.com) and the hidden **.git** folder (GitHub's own records).

5. **Copy in the new files**

   Open the **website** folder from the zip, select everything inside it, and copy it into the repository folder. The hidden **.nojekyll** file must come too: in Finder press **Cmd + Shift + . (period)** to show hidden files, then select all again. The repository folder should now hold **index.html**, **products.json**, the **img** folder and **.nojekyll**, plus **CNAME** if you had one.

6. **Commit and push**

   Go back to GitHub Desktop. The changes are listed on the left. Type **New Yonge Jewelry site** in the **Summary** box at the bottom left, click **Commit to main**, then click **Push origin** at the top.

7. **Turn on GitHub Pages**

   On github.com, open the repository and go to **Settings › Pages**. Under **Build and deployment**, set **Source** to **Deploy from a branch**, **Branch** to **main** and the folder to **/ (root)**, then click **Save**. If you use your own domain, check that it appears under **Custom domain** and tick **Enforce HTTPS** once it is available.

> **Check.** Wait about two minutes, then open your site and reload it. You should see the new shop with photos and prices. Browsing, the cart, "See it in store" and the text buttons already work. If you try "Buy online", the payment step says online payment is not switched on yet. That is expected until Part 4.

PART 2 OF 6 · ABOUT 10 MINUTES

Create the Square app

Square needs to know your website is allowed to take payments for the shop. You do that by creating a free app in Square's Developer Console. Write the IDs on the worksheet at the end of this guide as you go.

1. **Sign in to the Developer Console**
   Go to **developer.squareup.com** and sign in with the shop's Square account, the same login used for Square at the counter.

2. **Create the app**
   Open **Applications** (the Developer Console) and create a new application. Name it **Yonge Jewelry website**.

3. **Copy the Sandbox Application ID**
   Open the app, then **Credentials**. Make sure the switch at the top says **Sandbox**. Copy the **Sandbox Application ID**. It starts with `sandbox-sq0idb-`.

4. **Copy the Sandbox Access token**
   On the same page, show and copy the **Sandbox Access token**. Save it in a password manager or a private note for Part 3. Do not write it on the worksheet.

5. **Copy the Sandbox Location ID**
   Open **Locations** in the left menu (still in Sandbox) and copy the **Location ID**.

> **Check.** You now have three things: the Sandbox Application ID and Location ID on the worksheet, and the Sandbox Access token saved privately.

Why sandbox?

Sandbox credentials only work with test cards, so you can try the whole checkout without moving real money. Part 6 swaps them for the Production credentials.

PART 3 OF 6 · ABOUT 15 MINUTES

Set up the payment helper on Cloudflare

The payment helper is the file payments-worker.js. Cloudflare runs it on its free Workers plan, which allows up to 100,000 requests a day. That is far more than a shop needs.

1. **Create a free Cloudflare account**

   Go to **dash.cloudflare.com/sign-up** and create an account. You do not need to move your domain to Cloudflare.

2. **Create a Worker**

   In the left menu, open **Workers & Pages** (it may be under **Compute**). Click **Create**, choose to start with **Hello World**, name it **yonge-payments** and click **Deploy**.

3. **Paste the payment code**

   Click **Edit code**. Select all of the sample code in the editor and delete it. Open **payments-worker.js** from the zip in VS Code (or Notepad on Windows), copy all of it, paste it into the Cloudflare editor, then click **Deploy**.

4. **Add the settings**

   Go back to the Worker and open **Settings › Variables and Secrets**. Click **Add** for each of these, then click **Deploy** (or **Save**):

Name	Type	Value
`SQUARE_ACCESS_TOKEN`	**Secret**	The Sandbox Access token from Part 2
`SQUARE_LOCATION_ID`	Text	The Sandbox Location ID
`SQUARE_ENVIRONMENT`	Text	`sandbox`
`SITE_URL`	Text	Your website address, for example `https://yongejewelry.com`, with no slash at the end. If your site is on a github.io address, use that full address.

   Optional: if the site is reached at more than one address (for example both yongejewelry.com and the github.io address), add `ALLOWED_ORIGINS` as Text with the extra addresses separated by commas. The www version of SITE_URL is allowed automatically.

5. **Copy the Worker's address**

   The Worker's overview page shows its address, something like `https://yonge-payments.yourname.workers.dev`. Copy it onto the worksheet.

> **Check.** Open your Worker's address followed by `/stock` in a browser, for example `https://yonge-payments.yourname.workers.dev/stock`. You should see `{"sold":{}}`. If you see an error instead, check the four settings (see Troubleshooting).

PART 4 OF 6 · ABOUT 5 MINUTES

Connect the website to Square

One small edit tells the website which Square app to use and where the payment helper lives.

1 Open index.html in a code editor

In the repository folder from Part 1, open **index.html** with VS Code (free at code.visualstudio.com). Do not use TextEdit or Word: they can change the file without telling you.

2 Find the Square settings

Press **Cmd + F** (Windows: **Ctrl + F**) and search for `var SQUARE`. You will see:

```
var SQUARE={
  applicationId:'',
  locationId:'',
  environment:'sandbox',
  apiBase:''
};
```

3 Fill in your three values

Type them between the quote marks, keeping the quote marks and commas exactly as they are. It should look like this, with your own values:

```
var SQUARE={
  applicationId:'sandbox-sq0idb-YOUR-ID',
  locationId:'YOUR-LOCATION-ID',
  environment:'sandbox',
  apiBase:'https://yonge-payments.yourname.workers.dev'
};
```

Save the file.

4 Commit and push

In GitHub Desktop, type **Connect Square sandbox** in the Summary box, click **Commit to main**, then **Push origin**. Wait about two minutes.

> **Important.** Only the Application ID, Location ID and Worker address go in index.html. These are safe to be public. The access token never goes here.

PART 5 OF 6 · ABOUT 10 MINUTES

Test with a practice card

Now you buy something with a Square test card. Nothing is charged, but the whole checkout runs for real.

1. **Open your site in a private window**
   A private (incognito) window makes sure you get the newest version of the site.

2. **Go to checkout**
   Add a piece to the cart, click **Buy online**, fill in your details (use your own email) and click **Continue to payment**. Square's card form appears under the amount due.

3. **Enter a Square test card**
   In Square's developer documentation, search for **Sandbox payments** or **sandbox test values**. Use one of the test card numbers listed there, any future expiry date, and the security code and postal code that page gives.

4. **Pay**
   Click **Pay**. You should see **Thank you. Your order is confirmed.** and a link to the Square receipt.

5. **Find the order in Square**
   In the Developer Console, open **Sandbox test accounts** and open the test seller's Square Dashboard. The order is there with its pieces, shipping and tax.

> **Check.** If the piece you bought was the last one in stock, it now shows Sold on your site. Practice orders only count while the site is in sandbox. They stop counting when you switch to production.

PART 6 OF 6 · ABOUT 10 MINUTES

Go live with real payments

Going live means swapping the three sandbox values for production ones, in Square, Cloudflare and index.html.

1. **Get the Production credentials**

   In the Square Developer Console, open your app's **Credentials** and switch to **Production**. Copy the **Production Application ID** (it starts with **`sq0idp-`**) and the **Production Access token**. Then copy the production **Location ID** from **Locations**. Square may ask you to confirm account details first.

2. **Update Cloudflare**

   In your Worker's **Settings › Variables and Secrets**, replace **`SQUARE_ACCESS_TOKEN`** with the production token and **`SQUARE_LOCATION_ID`** with the production location ID, and set **`SQUARE_ENVIRONMENT`** to **`production`**. Click **Deploy**.

3. **Update index.html**

   Put in the production Application ID and Location ID, and change the environment:

   ```
   applicationId:'sq0idp-YOUR-PRODUCTION-ID',
   locationId:'YOUR-PRODUCTION-LOCATION-ID',
   environment:'production',
   ```

   Save, then in GitHub Desktop commit **Go live with Square** and push.

4. **Make one real purchase**

   Buy an inexpensive piece with your own card. Check that it appears in the shop's real Square Dashboard, then refund it there.

5. **Put the refunded piece back in stock**

   Refunds are not added back automatically, so the site still counts that piece as sold. Raise its qty by 1 in products.json (see Everyday tasks) to put it back.

> **Check.** You are live. Customers can now pay by card, orders land in your Square Dashboard, and pieces switch to Sold by themselves as stock runs out.

AFTER LAUNCH

Everyday tasks

All product details live in products.json, one piece per line. Open it in VS Code from the repository folder, make the change, save, then commit and push in GitHub Desktop. The site updates in about two minutes.

What a product line looks like

```
{"id": 12, "name": "VVS Clarity Lab-Grown Emerald Cut Diamond Necklace",
 "category": "Necklaces", "material": "925 Silver & 18K Vermeil", "qty": 1, "price": 2850},
```

The **id** is the number from the # column of your product sheet, and the photo is **img/products/12.jpg**. **category** is one of Rings, Earrings, Necklaces or Bracelets.

Task	What to do
You sold a piece in the shop	Search for its number, for example **"id": 12,**, and lower **qty** by one. Online sales are counted automatically, but sales at the counter are not.
Change a price	Change the number after **"price":**. Leave out the dollar sign and commas.
Mark a piece price on request	Set **"price": null**. It shows "Price on request" and cannot be bought online.
Restock a piece	Raise **qty**. A piece that showed Sold becomes available again.
Add a new piece	Copy a line, change the id to an unused number, then edit the name, category, material, qty and price. Add its photo as **img/products/<id>.jpg**; portrait photos about 465 by 600 pixels look best.
Remove a piece	Delete its whole line. Every line ends with a comma except the last one before the closing].

> **Tip.** If the shop shows "The collection could not load" after an edit, a comma, quote mark or bracket is missing in products.json. Undo the change in GitHub Desktop (right-click the file › Discard changes) and try again.

Held back from the product sheet

These pieces were left off the site until they are checked. Add their lines and photos when ready.

79 pieces	In stock but no photo yet
6 pieces	Brand or authenticity to verify: 61, 102, 128, 178, 246, 284
7 pieces	Diamond origin to confirm: 289, 291, 292, 293, 294, 295, 296
Changed	Duplicates merged: 288 into 118 (qty 2), 274 into 214 (qty 4). Moved to Rings: 93, 103, 195

IF SOMETHING GOES WRONG

Troubleshooting

What you see	What to check
The old site is still showing	Wait a few minutes and reload with Cmd + Shift + R (Windows: Ctrl + F5). In the repository, Settings › Pages should say your site is live.
"The collection could not load"	products.json must sit next to index.html. If you just edited it, look for a missing comma, quote mark or bracket.
Photos are missing	The img folder must be next to index.html, with the photos in img/products named like 12.jpg.
"Online payment is not switched on yet"	In index.html all three of applicationId, locationId and apiBase must be filled in. Push the change, then reload the site.
"The card form could not load"	The Application ID and environment must match: sandbox IDs with sandbox, production IDs with production. Ad blockers can block Square; try a private window.
"The payment did not go through"	Check the Worker's settings, especially SITE_URL (exact address, https, no slash at the end), and that the token and location ID come from the same mode. Your Worker's Logs page in Cloudflare shows the error.
The /stock check shows an error	A Worker setting is missing or the token and location ID are from different modes. Deploy again after changing settings.
Paying works on one address but not another	Add the other address (for example the github.io one) to ALLOWED_ORIGINS in the Worker settings.
A piece shows Sold but you still have it	Raise its qty in products.json. qty is the stock you started with; online sales are subtracted from it.
"Card declined" during sandbox testing	Use the test card numbers from Square's sandbox page. Real cards do not work in sandbox.

KEEP THIS PAGE

Worksheet

Write these down as you go. They are not secret, so this page is safe to keep with the shop's paperwork.

Website address
SITE_URL, e.g. https://yongejewelry.com _____

GitHub account and repository _____

Cloudflare Worker address
apiBase, e.g. https://yonge-payments.yourname.workers.dev _____

Sandbox Application ID
starts with sandbox-sq0idb- _____

Sandbox Location ID _____

Production Application ID
starts with sq0idp- _____

Production Location ID _____

> **Important.** Do not write access tokens on this page. Keep them in a password manager. If you think a token has leaked, replace it in the Square Developer Console and update SQUARE_ACCESS_TOKEN in Cloudflare.

Shop details used on the site

Address	29 St Andrew St, Unit B, Toronto, ON M5T 1K7
Phone (text buttons)	437-212-6273
Hours	Open daily, 12 pm to 6 pm
Shipping	Ontario free; rest of Canada $18 (free from $150); US $28 and Europe $45 (free from $250)